RESEARCH & PROTOTYPE REVIEW

Flowchart Report

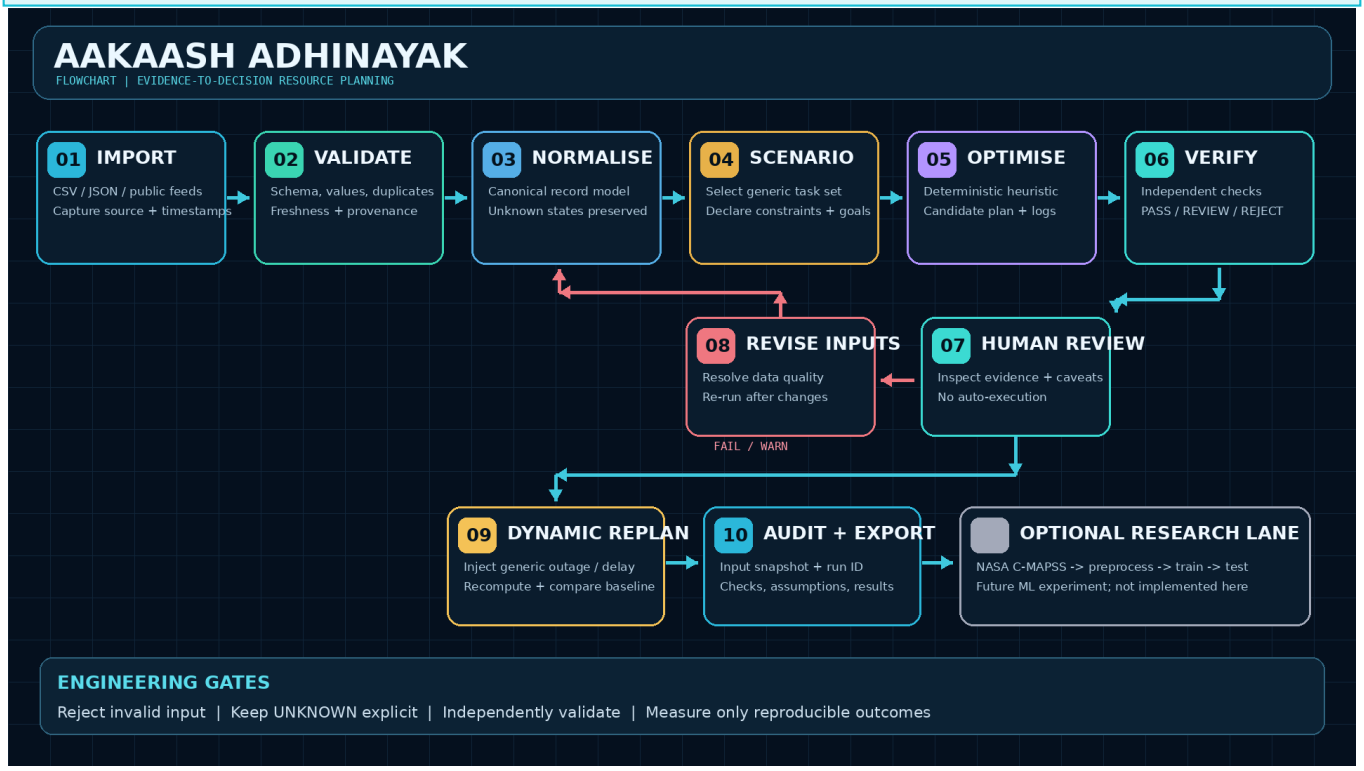
End-to-end workflow, branching, evidence objects and acceptance gates.

SIH 2026 | PS SIH26250 | Alchemy Architects

Purpose and reading guide

This flowchart describes the intended evidence-to-decision workflow for a generic, non-operational planning prototype. It is a technical target flow that combines modules present in the supplied source with explicit guardrails and future-work boundaries. It is not a claim that every stage is already implemented or production-ready.

Use in PowerPoint: insert the separate 1920 x 1080 PNG for a slide; use the SVG when a vector graphic is preferred. The diagram deliberately marks the ML lane as a future experiment.



1. Main workflow and decision gates

Stage	Input	Operation	Decision / output
1. Import	CSV/JSON, approved public source, synthetic fixture	Parse and retain source metadata.	Reject malformed data; retain provenance and retrieval time.
2. Validate	Parsed records	Check schema, types, duplicates, bounds, timestamps and source class.	Valid, warning, quarantined or rejected.
3. Normalise	Validated records	Map field names and units to a canonical schema; preserve UNKNOWN.	Versioned normalized snapshot.
4. Scenario	Resource/task records	Select generic tasks, objectives and constraints.	Scenario ID and declared assumptions.
5. Optimise	Scenario snapshot	Run deterministic heuristic to choose candidate slots.	Assignments plus unassigned tasks, objective and logs.
6. Verify	Candidate plan	Independent checks for readiness, windows, overlaps, capabilities, dependencies and freshness warning.	PASS, REVIEW or REJECT; human review remains mandatory.
7. Revise	Failed checks or warnings	Correct source data or constraints and rerun.	No silent acceptance of invalid plans.
8. Human review	Validated candidate + evidence	Inspect source freshness, assumptions, unassigned tasks and trade-offs.	Human-reviewed proposal only.
9. Replan	Controlled generic disruption	Change a demo availability/weather-delay input and rerun.	Compare baseline against new candidate on identical metrics.
10. Audit/export	Input snapshot, results, validator output	Record run identifier, model/code version, source and decision rationale.	Machine-readable evidence bundle.

2. Important branches

- Import/schema failure -> reject file with row/field explanation; do not quietly coerce unknown values into available/valid states.
- Readiness unknown, disputed or under maintenance -> resource excluded from allocation; preserve reason and source timestamp.
- No feasible slot -> task is explicitly unassigned. The system must not fabricate a resource or silently violate a constraint.
- Independent validation finds a fatal issue -> candidate status is REJECTED and must return to correction/replanning.
- Only warnings or unassigned tasks remain -> REVIEW REQUIRED, not a green “success” indicator.
- A disruption is introduced -> capture baseline, apply one explicit change, rerun the same scenario, validate independently and compare deltas.
- Predictive maintenance lane -> use only after the exact NASA dataset is downloaded, permitted, preprocessed and evaluated; current UI metrics alone are not a trained ML pipeline.

3. Data and evidence objects

Recommended minimum objects are ResourceSnapshot, TaskSnapshot, SourceRecord, DataQualityIssue, ScenarioDefinition, OptimizationRun, Assignment, ValidationReport, DisruptionEvent, HumanReview, AuditEvent and ExportManifest. Each export should carry a schema version, timestamp, input hash computed from actual serialized data, software version and source provenance.

4. Acceptance tests aligned with the flow

- Reject a malformed CSV/JSON input and report the precise row/field.
- Verify that UNKNOWN, UNAVAILABLE and MAINTENANCE resources cannot be assigned.
- Reject double-booking, unqualified assignment, availability-window violation, unsatisfied dependency and out-of-deadline schedule.
- Run a feasible and an infeasible dataset and account for every task as assigned or unassigned.

- Repeat the same input and confirm deterministic output aside from generated run metadata.
- Replay an identical disruption twice and retain before/after records and validation outcomes.
- Export an audit bundle containing both successful and failed checks.

References and source notes

[1] AI Studio application URL
https://aakaash-adhinayak.ai.studio/
User-supplied hosted prototype link. Automated retrieval was unsuccessful in this review; screenshots and the exported source bundle were inspected instead.

[2] AviationWeather.gov Data API
https://aviationweather.gov/data/api/
Official public civil aviation weather data API documentation; connector status and terms must be verified before live claims.

[3] NASA Prognostics Center of Excellence Dataset Repository
https://www.nasa.gov/intelligent-systems-division/discovery-and-systems-health/pcoe/pcoe-data-set-repository/
Official NASA repository lists turbofan engine degradation simulation data and the C-MAPSS benchmark. It is not military telemetry.

[4] Google OR-Tools CP-SAT documentation
https://developers.google.com/optimization/cp/cp_solver
Reference for a future exact constraint-programming engine; the inspected implementation uses a deterministic heuristic rather than OR-Tools.